```
from Crypto.Cipher import AES

key = b'mysecretencryptionkey'
cipher = AES.new(key, AES.MODE_EAX)
ciphertext, tag = cipher.encrypt_and_digest(b'Sensitive Data')
print(f"Encrypted Data: {ciphertext}")
```

2. **Data Masking**:
    - **Purpose**: Replace sensitive data with placeholders during processing.
    - **Example Code**:

```
def mask_data(data, mask_char="*"):
    return mask_char * len(data)

print(mask_data("123-45-6789")) # Output: *********
```

3. **Secure APIs**:
    - **Techniques**: Implement HTTPS for secure communication and use authentication mechanisms like OAuth to validate API requests.
    - **Frameworks**: Use Python frameworks like Flask or FastAPI with security plugins to ensure API robustness.
4. **Federated Learning**:
    - **Definition**: Train models locally on user devices to avoid transmitting raw data to centralized servers.
    - **Advantages**: Reduces the risk of data breaches and ensures compliance with data localization laws.
5. **Privacy-Preserving Techniques**:
    - **Differential Privacy**:
        - Adds noise to datasets to prevent identification of individual records.
        - **Example Code**:

```
import numpy as np

def add_differential_noise(data, epsilon=1.0):
    noise = np.random.laplace(0, 1/epsilon, size=len(data)) return
data + noise
```

    - **Homomorphic Encryption**: